

Cấu trúc dữ liệu và giải thuật

Ngăn xếp (Stack)

Hàng đợi (Queue)

Ngăn xếp (Stack)

- **Stack (ngăn xếp):** là cấu trúc dữ liệu trừu tượng chứa các đối tượng làm việc theo cơ chế **LIFO** (*Last In First Out*), tức việc thêm 1 đối tượng vào Stack hoặc lấy 1 đối tượng ra khỏi Stack được thực hiện theo cơ chế “**vào sau ra trước**”.
- **Stack** là một cấu trúc dạng thùng chứa các đối tượng, trong đó có 2 thao tác chính là : Push và Pop.

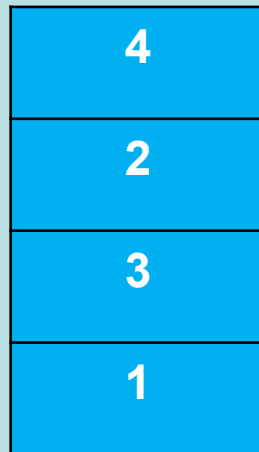
Các thao tác trên Stack

- **Push(p)**: Thêm đối tượng p vào Stack.
- **Pop()**: Lấy đối tượng từ Stack.
- **isEmpty()**: Kiểm tra Stack có rỗng hay không?
- **Top()**: Trả về giá trị của phần tử nằm đầu Stack mà không hủy nó khỏi Stack.

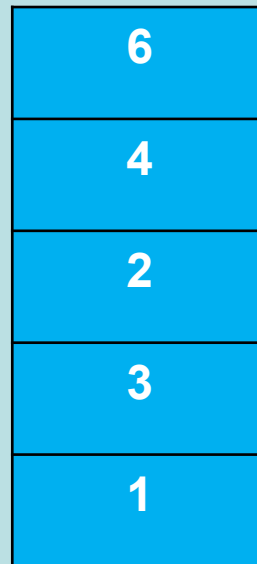
Ngăn xếp (Stack)

Push(6)

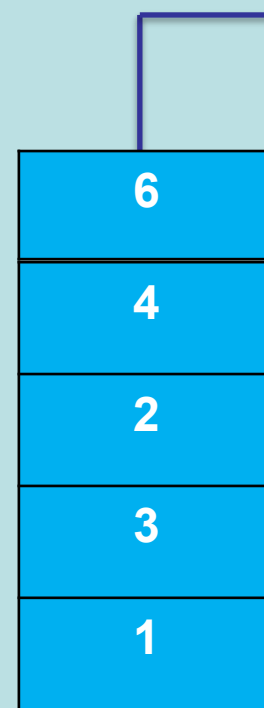
6



Top() → 6

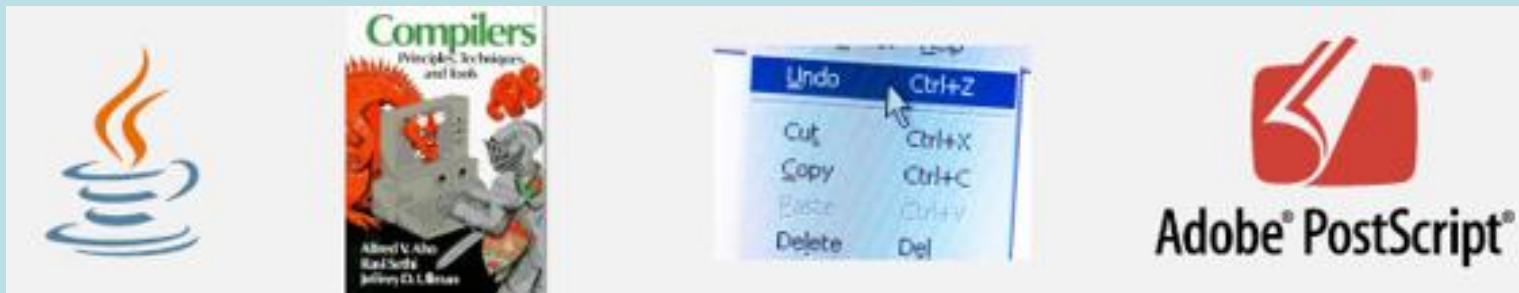


Pop()



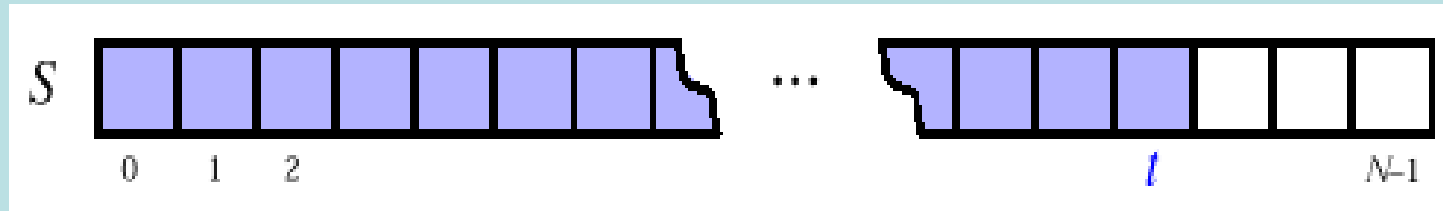
Ngăn xếp L

- Máy ảo Java
- Parsing trong trình biên dịch
- Undo trong Word
- Nút Back trong trình duyệt Web
- Cài đặt / thực hiện lời gọi hàm trong trình biên dịch
- ...



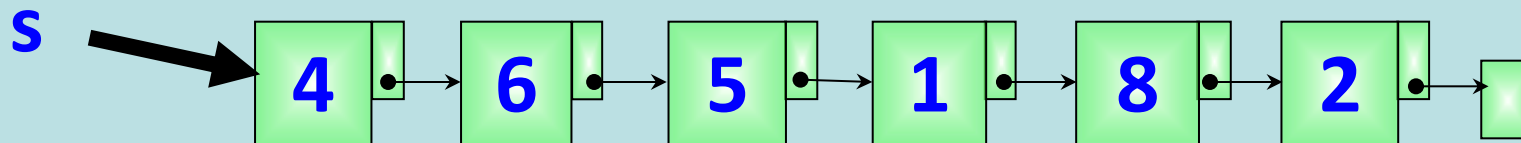
Cài đặt Stack

➤ Dùng mảng 1 chiều



Data S [N];
int t;

➤ Dùng danh sách liên kết đơn



List s

❖ Thêm và hủy cùng phía

Cài Stack bằng DSLK đơn

- Kiểm tra tính rỗng của Stack

```
int IsEmpty(LIST s)
{
    if (s.pHead == NULL) //Stack rỗng
        return 1;
    else
        return 0;
}
```

Thêm 1 phần tử vào Stack

```
void Push(LIST &s, Node *Tam)
{
    if (s.pHead == NULL)
    {
        s.pHead = Tam;
        s.pTail = s.pHead;
    }
    else
    {
        Tam->pNext = s.pHead;
        s.pHead = Tam;
    }
}
```


Lấy 1 phần tử từ Stack

```
int Pop(LIST &s, int &t)
{ Node *p;
  if(IsEmpty(s) != 1)
  {
    if (s.pHead != NULL)
    {
      p = s.pHead;
      t = p->Info;
      s.pHead = s.pHead->Next;
      if (s.pHead == NULL) s.Tail = NULL;
      return 1;
      delete p;
    }
  }
  return 0;
}
```

Lấy giá trị từ đỉnh Stack

```
int Top(LIST s)
{
    if(IsEmpty(s) != 1)
        return s.pHead->info;
    return 0;
}
```

Hàng đợi (Queue)

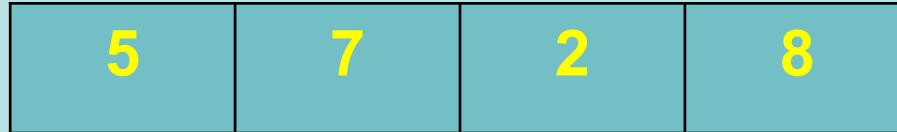
- **Queue (hàng đợi):** là 1 vật chứa các đối tượng làm việc theo cơ chế **FIFO** (*First In First Out*), tức việc thêm 1 đối tượng vào Queue hay lấy 1 đối tượng ra khỏi Queue thực hiện theo cơ chế “**vào trước ra trước**”.
- **Queue** là một cấu trúc dạng ống có 2 đầu, dữ liệu vào ở một đầu và được lấy ra ở đầu kia. Thao tác chính trên queue là EnQueue và DeQueue: thêm vào và lấy đối tượng ra khỏi hàng đợi.

Các thao tác trên Queue

- **EnQueue(p)**: Thêm đối tượng p vào cuối hàng đợi.
- **DeQueue()**: Lấy đối tượng ở đầu hàng đợi
- **isEmpty()**: Kiểm tra hàng đợi có rỗng hay không?
- **Front()**: Trả về giá trị của phần tử nằm đầu hàng đợi mà không hủy nó.

Các thao tác trên Queue

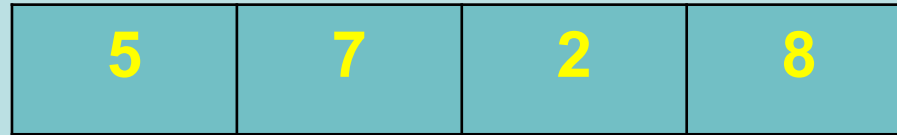
Hàng đợi Q



EnQueue(9)



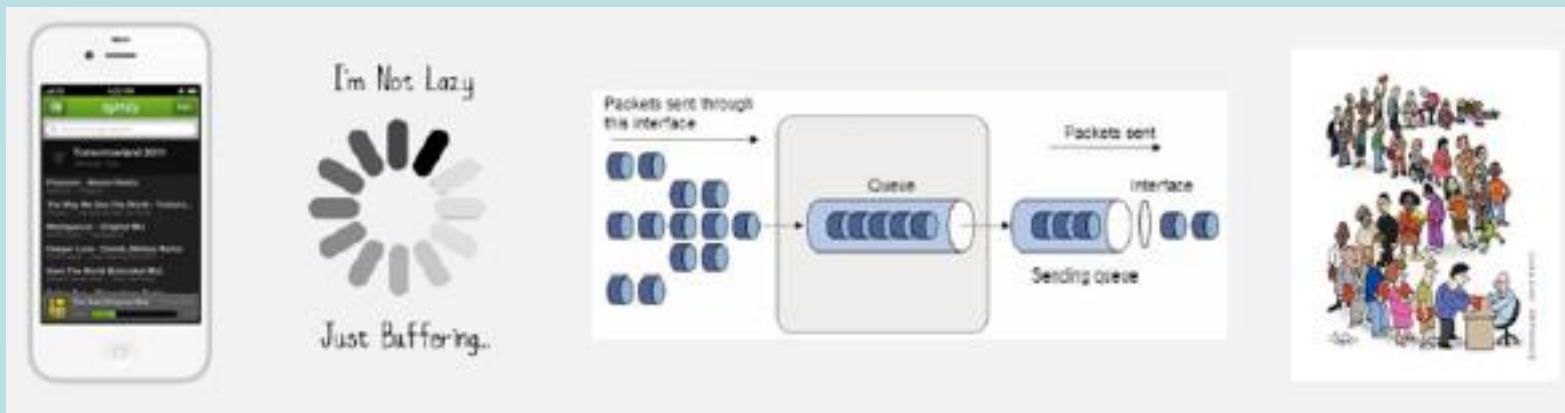
Front() → 8



DeQueue()

Một số ứng dụng của hàng đợi

- Data buffer
- Hàng đợi xử lý trong các trung tâm chăm sóc khách hàng
- Xếp hàng thanh toán trong siêu thị
- Xếp hàng chơi trò chơi trong các khu vui chơi giải trí
- ...



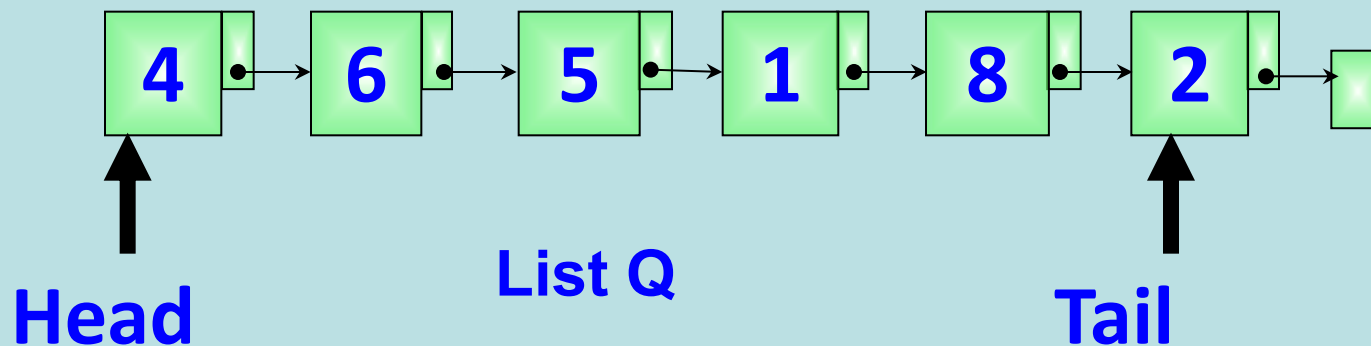
Cài đặt Queue

- Dùng mảng 1 chiều



Data S [N];
int f,r;

- Dùng danh sách liên kết đơn



* Thêm và hủy khác phía

Cài đặt Queue bằng DSLK

- Kiểm tra Queue có rỗng?

```
int IsEmpty(LIST &Q)
{
    if (Q.pHead == NULL) //Queue rỗng
        return 1;
    else
        return 0;
}
```


Thêm 1 phần tử vào Queue

```
void EnQueue(LIST &Q, Node *tam)
{ if (Q.pHead == NULL)
    { Q.pHead = tam;
      Q.pTail = tam;
    }
  else
    { Q.pTail->Next = tam;
      Q.pTail = tam;
    }
}
```

Lấy 1 phần tử từ Queue

```
int DeQueue(LIST &Q, int &trave)
{  Node *p;
    if (IsEmpty(Q) != 1)
        if (Q.pHead != NULL)
            {
                p = Q.pHead;
                trave = p->Info;
                Q.pHead = Q.pHead->Next;
                if(Q.pHead == NULL)    Q.pTail = NULL;
                return 1;
                delete p;
            }
    return 0;
}
```

Bài tập 1

1. Hãy cho biết nội dung của stack sau mỗi thao tác trong dãy:

EAS*Y**QUE***ST***I*ON

Với một kí tự tượng trưng cho thao tác thêm chữ cái tương ứng vào stack, dấu * tượng trưng cho thao tác lấy nội dung một phần tử trong stack in lên màn hình. Hãy cho biết sau khi hoàn tất chuỗi thao tác, những gì xuất hiện trên màn hình?

2. Cài đặt chương trình cho phép thực hiện các phép tính +,-,*,/ trên các số có tối đa 30 chữ số,

có chức năng nhớ (M+, M-, MC, MR).

3. Viết chương trình thực hiện các thao tác trên đa thức.

4. Hãy viết chương trình mô phỏng cho bài toán “Tháp Hà Nội” bằng cách sử dụng ngăn xếp.

5. Viết chương trình tìm tất cả các cặp dấu ngoặc tương ứng trong một chương trình viết bằng ngôn ngữ C/C++.

Bài tập 2

Vấn đề : Cài đặt một queue sử dụng 2 stack ?

Ứng dụng :

- Job interview
- Cài đặt queue ưu tiên
- ...



Bài tập

1. Hãy cho biết nội dung của stack sau mỗi thao tác trong dãy:

EAS*YQUE***ST***I*ON**

Với một kí tự tượng trưng cho thao tác thêm chữ cái tương ứng vào stack, dấu * tượng trưng cho thao tác lấy một phần tử trong stack và xuất lên màn hình. Hãy cho biết sau khi hoàn tất chuỗi thao tác, những gì xuất hiện trên màn hình?

2. Giả sử cho hàm push(a) là hàm thực hiện nạp a vào ngăn xếp và hàm pop() là hàm lấy phần tử ra khỏi ngăn xếp. Cho dãy các thao tác sau đây, biết rằng ngăn xếp ban đầu được khởi tạo rỗng:

push(5), push(3), pop(), push(2), push(8), pop(), pop(), pop(), push(9), push(1), pop(), push(7), push(6), pop(), pop(), push(4), pop(), pop().

Hãy viết ra dãy phần tử của ngăn xếp (chỉ rõ vị trí đầu ngăn xếp) sau khi thực hiện mỗi thao tác (mỗi thao tác ghi trên 1 dòng)

3. Giả sử cho hàm enq(a) là hàm thực hiện nạp a vào hàng đợi và hàm deq() là hàm thực hiện lấy phần tử ra khỏi hàng đợi. Cho dãy các thao tác sau đây, biết rằng hàng đợi ban đầu được khởi tạo rỗng:

enq(5), enq(3), deq(), enq(2), enq(8), deq(), enq(9), enq(1), deq(), enq(7), enq(6), deq(), deq(). enq(4), deq(), deq().

Hãy viết ra dãy các phần tử của hàng đợi (chỉ rõ vị trí đầu và cuối của hàng đợi) sau khi thực hiện mỗi thao tác (mỗi thao tác ghi trên 1 dòng)

Biểu thức trung tố - hậu tố

Biểu thức trung tố - hậu tố

- **Biểu thức trung tố (infix expression)** là cách viết biểu thức mà **toán tử nằm giữa hai toán hạng** — đây là dạng quen thuộc nhất mà chúng ta dùng trong toán học hằng ngày.
- Ví dụ:

$$3 * 5$$

$$(2 + 5) * 7 - 3$$

$$A * (B + C)$$

Biểu thức trung tố - hậu tố

- **Biểu thức hậu tố (postfix expression)** là cách viết biểu thức mà **toán tử nằm sau các toán hạng** (còn gọi là ký pháp Ba Lan ngược – Reverse Polish Notation (RPN), do Jan Lukasiewicz phát triển).
- Ví dụ:
 $3\ 5\ *$
 $2\ 5\ +\ 7\ *\ 3\ -$
 $A\ B\ C\ +\ *$

Biểu thức trung tố - hậu tố

- **Biểu thức hậu tố** không chứa các dấu ngoặc, nó chỉ gồm các toán hạng và các dấu phép toán.
- Việc tính giá trị của biểu thức ở dạng biểu thức hậu tố được thực hiện: biểu thức được đọc từ trái sang phải cho đến khi tìm được một toán tử. Tại thời điểm đó, hai toán hạng cuối cùng được đọc kết hợp với toán tử này.

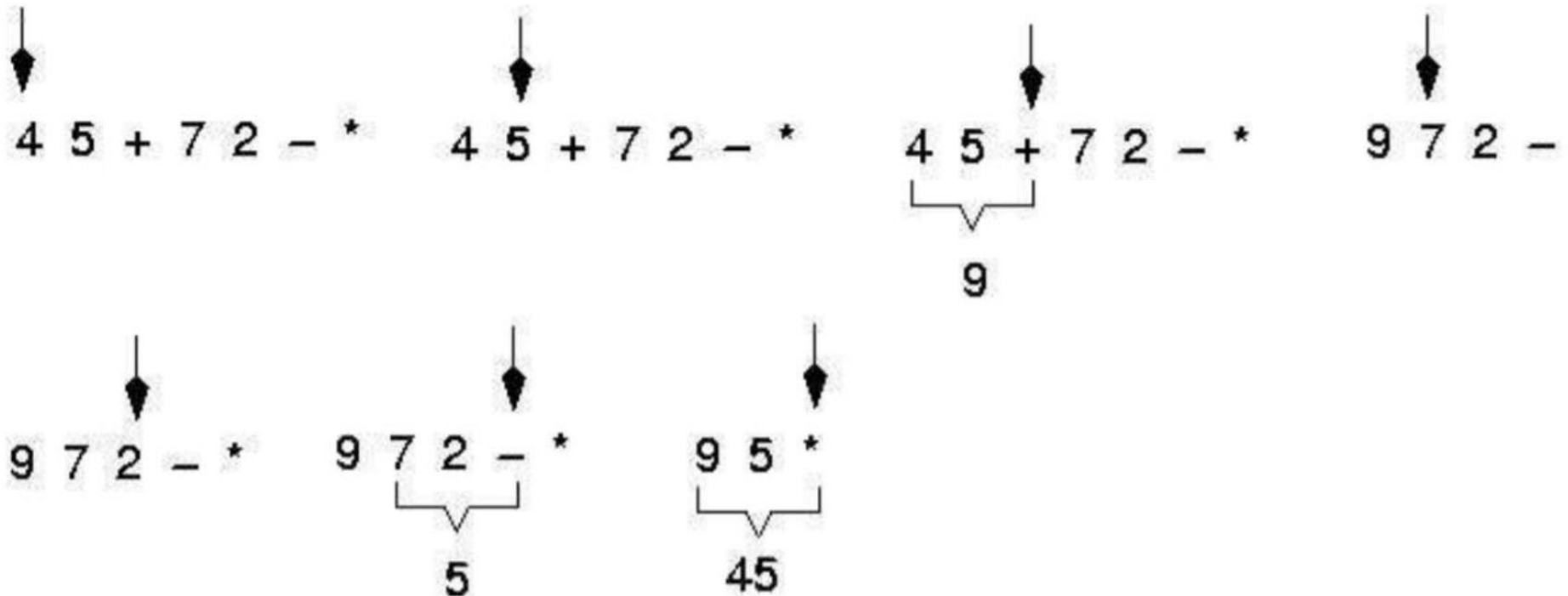
Biểu thức trung tố - hậu tố

Ưu điểm:

- Không cần dấu ngoặc.
- Không cần xét độ ưu tiên toán tử.
- Rất dễ tính toán bằng **stack**.
- Phù hợp cho máy tính xử lý.

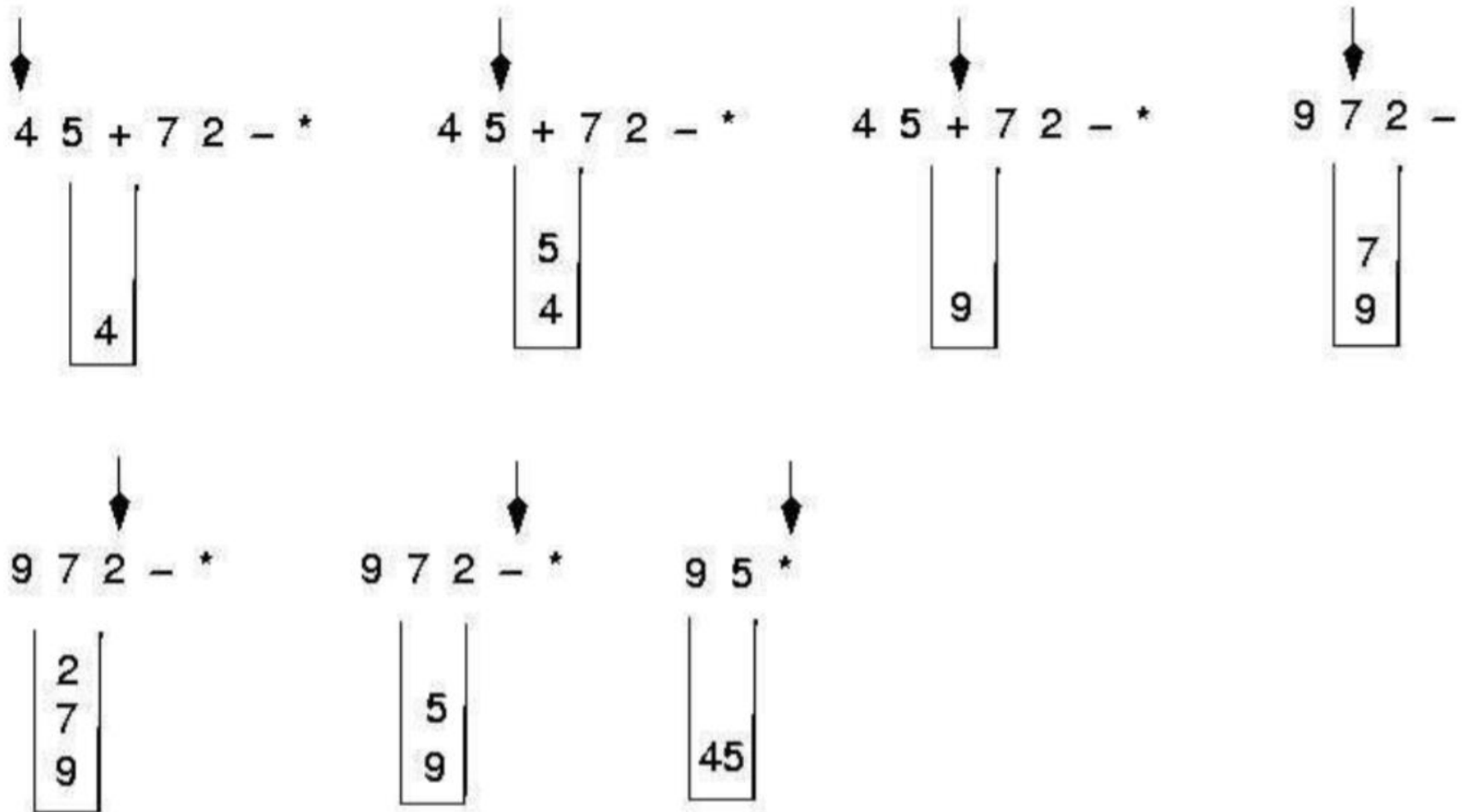
Biểu thức trung tố - hậu tố

Cách tính biểu thức hậu tố



Biểu thức trung tố - hậu tố

Cách tính biểu thức hậu tố sử dụng stack



Biểu thức trung tố - hậu tố

Tại sao cần phải chuyển biểu thức trung tố sang biểu thức hậu tố?

Đối với máy tính, khi xử lý biểu thức trung tố cần:

- Xét độ ưu tiên toán tử
- Xét dấu ngoặc nếu có

Dẫn đến việc xử lý phức tạp hơn.

Biểu thức trung tố - hậu tố

Biểu thức hậu tố làm tăng hiệu quả xử lý cho máy tính

Trong các hệ thống như:

- Trình biên dịch (compiler)
- Máy tính cầm tay
- Trình thông dịch (interpreter)

Biểu thức trung tố được chuyển sang hậu tố để:

- Giảm độ phức tạp
- Tăng tốc độ xử lý
- Dễ cài đặt

Biểu thức trung tố - hậu tố

- Thuật toán **Shunting-yard** là một phương pháp kinh điển được phát triển bởi Edsger Dijkstra để chuyển đổi biểu thức từ dạng **Trung tố** (Infix) sang dạng **Hậu tố** (Postfix).
- Thuật toán này sử dụng một **Ngăn xếp (Stack)** để giữ các toán tử và một **Hàng đợi (Queue)** để chứa kết quả đầu ra.

Biểu thức trung tố - hậu tố

Tạo:

- Stack rỗng (S)
- Chuỗi kết quả rỗng (P)

Bước 1:

Duyệt từng ký tự trong biểu thức trung tố:

1.1 Nếu là toán hạng (A, B, 1, 2...)

→ Thêm trực tiếp vào P

1.2 Nếu là dấu (

→ Đẩy vào stack

1.3 Nếu là dấu)

→ Pop từ stack ra P cho đến khi gặp ((loại bỏ (

Biểu thức trung tố - hậu tố

1.4 Nếu là toán tử (+ - * / ^)

Trong khi:

- Stack không rỗng.
- Nếu toán tử đỉnh stack có độ ưu tiên $\geq$ toán tử hiện tại
→ Pop từ stack ra P
- Sau đó push toán tử hiện tại vào stack.

Bước 2:

Sau khi duyệt hết biểu thức infix, nếu trong stack còn phần tử thì lấy tất cả ra và lần lượt cho vào P.

Ví dụ:

Infix: $((2 + 4) * 7) + 3 * (9 - 5)$

Postfix: $2\ 4\ +\ 7\ *\ 3\ 9\ 5\ -\ *\ +$

STT	Token	Stack	Output
1	(	(	
2	(	((	
3	2	((	2
4	+	((+	2
5	4	((+	2 4
6	)	(	2 4 +
7	*	(*	2 4 +
8	7	(*	2 4 + 7
9	)		2 4 + 7 *
10	+	+	2 4 + 7 *
11	3	+	2 4 + 7 * 3
12	*	+ *	2 4 + 7 * 3
13	(	+ * (	2 4 + 7 * 3
14	9	+ * (	2 4 + 7 * 3 9
15	-	+ * (-	2 4 + 7 * 3 9
16	5	+ * (-	2 4 + 7 * 3 9 5
17	)	+ *	2 4 + 7 * 3 9 5 -
18		Empty	2 4 + 7 * 3 9 5 - * +

Ví dụ: tính giá trị biểu thức hậu tố

$$2\ 4\ +\ 7\ *\ 3\ 9\ 5\ -\ *\ +$$

= 54

STT	Token	Stack
1	2	2
2	4	2 4
3	+	6
4	7	6 7
5	*	42
6	3	42 3
7	9	42 3 9
8	5	42 3 9 5
9	-	42 3 4
10	*	42 12
11	+	54
12		Empty